

WorldCup Hub

Documentation technique complète

Projet final — Building a Production-Ready Full Stack JS App
Application microservices : prédictions de matchs (ML), équipes favorites,
scores en direct de la Coupe du Monde 2026

Dépôt GitHub : github.com/ilyessee/worldcup-hub
Juillet 2026

1. Vue d'ensemble

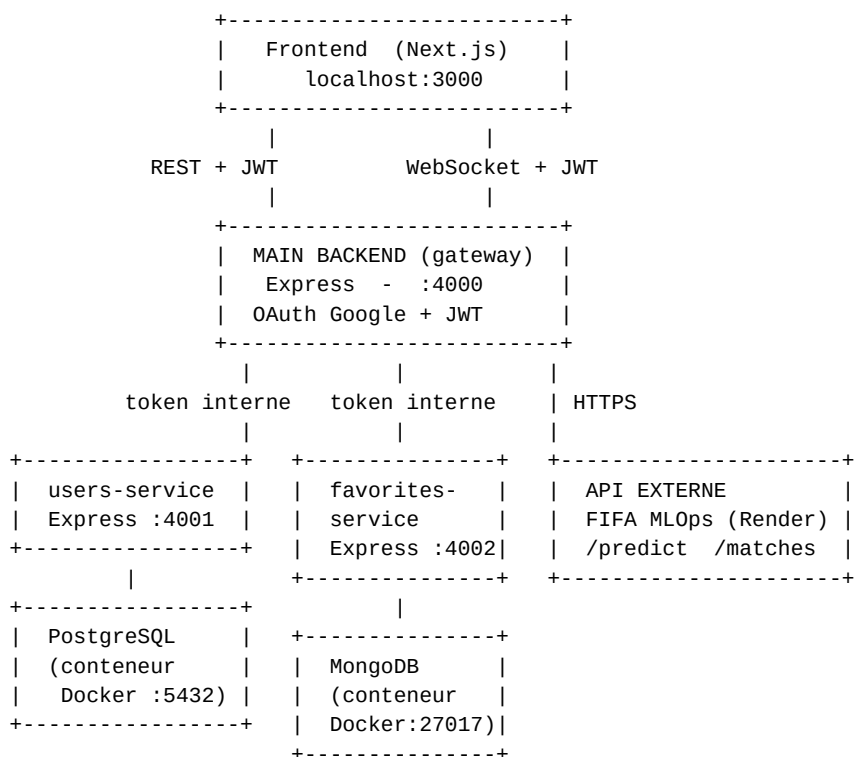
WorldCup Hub est une application web full stack JavaScript en **architecture microservices**. Elle permet à un utilisateur connecté via son compte Google de prédire le résultat de matchs de la Coupe du Monde 2026 (via un modèle de machine learning servi par une API externe), de gérer ses équipes favorites et de suivre les scores en direct.

Le projet répond à toutes les exigences du sujet : un backend principal accompagné de deux microservices, deux bases de données (PostgreSQL et MongoDB), une authentification Google avec routes protégées, une API consommable par des clients externes avec un jeton JWT, deux paradigmes d'API (REST et WebSocket), et l'intégration d'un service API externe (bonus).

Stack technique

Composant	Technologie	Port	Base de données
frontend/	Next.js (React 19)	3000	—
main-backend/	Node.js + Express (gateway, auth, WebSocket)	4000	—
services/users-service/	Node.js + Express	4001	PostgreSQL 16
services/favorites-service/	Node.js + Express	4002	MongoDB 7
API externe	FIFA MLOps API (FastAPI + MLflow, sur Render)	—	—

2. Architecture microservices



Rôle de chaque service

- **main-backend (gateway)** : point d'entrée unique. Il gère le login Google, émet et vérifie les JWT, expose l'API REST publique (/api/v1) et le WebSocket, et orchestre les appels vers les microservices et l'API externe. Le frontend ne parle jamais directement aux microservices.

- **users-service** : gère les comptes utilisateurs (création au premier login, mise à jour du profil). Persiste dans PostgreSQL.
- **favorites-service** : gère les équipes favorites et l'historique des prédictions de chaque utilisateur. Persiste dans MongoDB.
- **API externe FIFA MLOps** : notre autre projet (cours DevOps/MLOps), déployé en production sur Render, qui sert les prédictions du modèle ML depuis un registre MLflow, ainsi que les matchs en direct.

Phrase clé pour l'oral : Le gateway est le seul service exposé au monde extérieur. Les microservices sont inaccessibles directement : ils exigent un token interne partagé que seul le gateway connaît (défense en profondeur).

3. Les deux bases de données

3.1 PostgreSQL — users-service (données relationnelles)

La table **users** stocke les comptes : id, google_id (unique), email, nom, photo, date de création. Au premier login Google, l'utilisateur est créé automatiquement (« upsert » : INSERT ... ON CONFLICT DO UPDATE). Les logins suivants rafraîchissent le profil sans créer de doublon grâce à la contrainte d'unicité sur google_id.

id	google_id	email	name
1	dev-demo@worldcuphub.dev	demo@worldcuphub.dev	Dev User
21	115219798010658965986	ilyesse.essalihi78@gmail.com	Ilyesse E

3.2 MongoDB — favorites-service (données flexibles)

Deux collections : **favorites** (userId + équipe, avec index unique sur le couple pour empêcher les doublons) et **predictionhistories** (chaque prédiction faite : équipes, phase, résultat prédit et l'objet complet des probabilités renvoyé par le modèle ML).

```
// favorites                                // predictionhistories
{ userId: 1, team: "Morocco" }              { userId: 1,
  homeTeam: "France",
  awayTeam: "Argentina",
  prediction: "home_win",
  probabilities: { home_win: 0.42,
                  draw: 0.17,
                  away_win: 0.41 } }
```

3.3 Pourquoi deux bases différentes ?

- **PostgreSQL** pour les utilisateurs : données structurées et stables, contrainte d'unicité forte sur google_id, relations possibles à l'avenir (SQL).
- **MongoDB** pour les favoris et l'historique : documents flexibles (le champ probabilités est un objet imbriqué qui peut évoluer avec le modèle), pas de jointures nécessaires, écritures simples et fréquentes.

3.4 Où sont physiquement stockées les données ?

PostgreSQL et MongoDB ne sont pas des fichiers du projet : ce sont des **serveurs** qui tournent dans des **conteneurs Docker** (lancés par docker compose up -d). Un conteneur étant jetable, les données sont persistées dans des **volumes Docker** (worldcup-hub_pgdata, worldcup-hub_mongodata) : des dossiers gérés par Docker sur le disque de la machine hôte. On peut détruire et recréer les conteneurs, ou déplacer le dossier du projet : les données survivent.

```
$ docker volume ls                                $ docker compose up -d # lance les 2 DB
local worldcup-hub_pgdata      (48 Mo)
local worldcup-hub_mongodata   (318 Mo)
```

Phrase clé pour l'oral : Les bases tournent dans des conteneurs Docker et persistent dans des volumes sur la machine hôte. En production, on les remplacerait par des bases managées (Supabase, MongoDB Atlas) en changeant uniquement l'URL de connexion dans le .env — le code ne change pas (principe 12-factor).

4. Authentification Google (OAuth 2.0)

Le login utilise le flux « authorization code » d'OAuth 2.0. L'application ne voit **jamais** le mot de passe Google de l'utilisateur : elle reçoit un code temporaire qu'elle échange contre le profil.

1. Le navigateur clique "Sign in with Google"
-> GET http://localhost:4000/auth/google
2. Le gateway redirige vers l'écran de consentement Google
(avec client_id + redirect_uri + scope "openid email profile")
3. L'utilisateur choisit son compte et accepte
4. Google redirige vers /auth/google/callback?code=XXXX
5. Le gateway échange ce code contre un access_token
(POST https://oauth2.googleapis.com/token, avec le client_secret)
6. Le gateway récupère le profil (sub, email, name, picture)
7. Upsert de l'utilisateur dans users-service (PostgreSQL)
8. Le gateway signe un JWT et redirige vers le frontend :
http://localhost:3000/auth/callback?token=<JWT>
9. Le frontend stocke le token et l'envoie sur chaque requête

Les identifiants (GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET) viennent de la Google Cloud Console (écran de consentement en mode test — seuls les utilisateurs de la liste de test peuvent se connecter). Ils sont stockés dans le fichier .env, qui est **exclu de git** (.gitignore) : aucun secret ne part sur GitHub.

Protection des routes

- **Côté frontend** : la page /dashboard vérifie la présence du token au chargement ; sans token, redirection immédiate vers la page de login. Toute réponse 401 de l'API (token expiré) purge le token et redirige aussi.
- **Côté backend** : le middleware requireAuth protège toutes les routes /api/v1/* et /auth/me. C'est la vraie barrière de sécurité — la protection frontend n'est que du confort utilisateur.

5. Le JWT en détail

Un JSON Web Token est composé de trois parties encodées en base64url et séparées par des points : **header.payload.signature**.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJEsImVtYWlsIjoie... . 6PovUJUh6BC...
```

PARTIE 1 - header (algorithme)	PARTIE 2 - payload (claims)
<pre>{ "alg": "HS256", "typ": "JWT" }</pre>	<pre>{ "sub": 1, <- id de l'utilisateur "email": "demo@worldcuphub.dev", "name": "Demo", "iat": 1783707410, <- émis à (timestamp) "exp": 1783750610 <- expire 12 h après }</pre>
PARTIE 3 - signature = HMAC-SHA256(header + "." + payload , JWT_SECRET)	

Les points essentiels à comprendre

- Le JWT n'est **pas chiffré**, il est **signé**. Le payload est du simple base64 : n'importe qui peut le lire (jwt.io le decode). On n'y met donc jamais de donnée sensible.
- La signature rend le token **infalsifiable** : si on modifie le payload (ex. changer sub:1 en sub:21 pour usurper un compte), la signature ne correspond plus et le serveur rejette avec 401. Seul le serveur connaît JWT_SECRET.
- Le serveur ne stocke **aucune session** : il vérifie la signature à chaque requête. C'est ce qui rend l'API « stateless » et facilement scalable (n'importe quelle instance du gateway peut valider n'importe quel

token).

- Le token expire au bout de 12 h (claim exp) : un token volé a une durée de vie limitée.

Phrase clé pour l'oral : Le JWT n'est pas chiffré, il est signé : le serveur ne stocke aucune session, il vérifie simplement la signature avec son secret à chaque requête — c'est ce qui rend l'API stateless et scalable.

6. API as a Service (démonstration JWT)

L'API /api/v1 est conçue pour être consommée aussi par des clients externes (Postman, curl, une autre application) munis d'un JWT. Démonstration exigée par le sujet :

Sans token → 401 Access Denied

```
$ curl -X POST http://localhost:4000/api/v1/predict \
  -H "content-type: application/json" \
  -d '{"homeTeam":"France","awayTeam":"Argentina"}'

HTTP 401
{ "error": "Access denied",
  "message": "Missing JWT. Provide an Authorization: Bearer <token> header." }
```

Avec token → 200 + prédiction

```
$ curl -X POST http://localhost:4000/api/v1/predict \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1..." \
  -H "content-type: application/json" \
  -d '{"homeTeam":"France","awayTeam":"Argentina","stage":"GROUP_STAGE"}'

HTTP 200
{ "prediction": "home_win",
  "probabilities": { "home_win": 0.4226, "draw": 0.1688, "away_win": 0.4087 } }
```

La gestion d'erreurs est explicite : 401 pour token manquant/invalid/expiré, 400 pour paramètres manquants, 502 si l'API externe est indisponible. Dans Postman : onglet Authorization, type Bearer Token.

Endpoints exposés par le gateway

Méthode	Route	Auth	Description
GET	/health	—	État du service
GET	/auth/google	—	Démarre le login Google
GET	/auth/google/callback	—	Callback OAuth, émet le JWT
GET	/auth/me	JWT	Profil de l'utilisateur connecté
POST	/api/v1/predict	JWT	Prédiction ML (API externe) + log en historique
GET	/api/v1/matches	JWT	Matches de la Coupe du Monde en direct
GET / POST	/api/v1/favorites	JWT	Lister / ajouter une équipe favorite
DELETE	/api/v1/favorites/:team	JWT	Retirer une équipe favorite
GET	/api/v1/history	JWT	Historique des prédictions
WS	/ws/live?token=<jwt>	JWT	Push des scores en direct (toutes les 60 s)

7. Les deux paradigmes d'API

7.1 REST (requête / réponse)

Le client prend l'initiative : il envoie une requête HTTP (GET, POST, DELETE...) et reçoit une réponse JSON. Ressources identifiées par des URL, codes de statut standards (200, 201, 401, 404...). Utilisé pour tout le CRUD : favoris, historique, prédictions.

7.2 WebSocket (push serveur, temps réel)

Connexion persistante bidirectionnelle : le client se connecte une fois à `ws://localhost:4000/ws/live` et le **serveur pousse** les mises à jour des matchs toutes les 60 secondes, sans que le client redemande. La connexion est authentifiée aussi : le JWT est passé en paramètre et vérifié à la connexion (fermeture avec le code 4401 si invalide).

```
REST : client --- requête ---> serveur --- réponse ---> client  (1 aller-retour)
WS   : client === connexion persistante === serveur
      <--- push scores (60 s)
      <--- push scores (60 s) ...
```

Phrase clé pour l'oral : REST : le client demande. WebSocket : le serveur pousse. Le WebSocket évite le polling côté client et donne du vrai temps réel ; il est authentifié par JWT comme le reste de l'API.

8. Intégration d'une API externe (bonus)

Les prédictions et les matchs en direct sont fournis par notre **projet MLOps FIFA World Cup** (FastAPI + registre de modèles MLflow), déployé en production sur Render : fifa-backend-production.onrender.com. Le gateway le consomme en HTTPS comme n'importe quel service tiers (fichier `main-backend/src/services/fifaClient.js`).

C'est une vraie intégration inter-systèmes : deux projets indépendants, deux dépôts, deux stacks (JavaScript ici, Python là-bas), reliés par un contrat d'API HTTP.

9. Sécurité en profondeur

- **Niveau 1 — périmètre public** : JWT obligatoire sur `/api/v1` et le WebSocket (401 sinon).
- **Niveau 2 — réseau interne** : les microservices refusent tout appel sans le header `x-internal-token` (secret partagé connu du seul gateway). Un appel direct à `users-service` renvoie 403 Forbidden.
- **Niveau 3 — secrets** : tous les secrets (JWT_SECRET, client secret Google, token interne) vivent dans des fichiers `.env` exclus de git. Des `.env.example` documentés servent de modèle.
- **Niveau 4 — CORS** : le gateway n'accepte que les origines du frontend (localhost en développement).

```
$ curl http://localhost:4001/users/1          # appel DIRECT au microservice
HTTP/1.1 403 { "error": "Forbidden: internal service" }
```

10. Lancer le projet en local

```
# Prérequis : Node.js >= 20, Docker Desktop

# 1. Bases de données (PostgreSQL + MongoDB)
docker compose up -d

# 2. Fichiers d'environnement (puis remplir les valeurs)
cp main-backend/.env.example main-backend/.env
cp services/users-service/.env.example services/users-service/.env
cp services/favorites-service/.env.example services/favorites-service/.env

# 3. Lancer les services (4 terminaux) - le .env est chargé automatiquement
cd services/users-service      && npm install && npm start    # :4001
cd services/favorites-service && npm install && npm start    # :4002
cd main-backend                && npm install && npm start    # :4000
cd frontend                    && npm install && npm run dev   # :3000

# 4. Ouvrir http://localhost:3000 et se connecter avec Google
```

Note : sans identifiants Google configurés, le mode développement propose un « Dev login » (`ALLOW_DEV_LOGIN=true`), pratique pour développer. À désactiver en production.

11. Questions / réponses probables à l'oral

Question	Réponse
Pourquoi une architecture microservices ?	Séparation des responsabilités : chaque service a un périmètre clair et sa propre base. On peut les développer, déployer et faire évoluer indépendamment (et les écrire dans des langages différents).
Que se passe-t-il si j'appelle un microservice directement ?	403 Forbidden : les microservices exigent le token interne que seul le gateway connaît. Toute la sécurité publique (JWT) est centralisée dans le gateway.

Question	Réponse
Différence entre JWT et session classique ?	Session : le serveur stocke l'état et donne un identifiant au client (stateful). JWT : tout est dans le token signé, le serveur ne stocke rien et vérifie la signature (stateless, scalable).
Pourquoi le mot de passe Google ne passe jamais par votre app ?	C'est le principe d'OAuth : l'utilisateur s'authentifie chez Google, qui nous renvoie un code temporaire échangé côté serveur contre son profil. Nous ne manipulons jamais ses identifiants.
Pourquoi deux bases de données différentes ?	PostgreSQL pour les données structurées avec contraintes fortes (users, unicité google_id) ; MongoDB pour les documents flexibles sans jointures (favoris, historique avec objet probabilités imbriqué).
Comment le frontend sait-il que le token a expiré ?	Toute réponse 401 de l'API déclenche la purge du token côté client et une redirection vers la page de login (géré dans le wrapper apiFetch).
Où sont stockées les données ?	Dans des volumes Docker sur la machine hôte (worldcup-hub_pgdata, worldcup-hub_mongodata). Les conteneurs sont jetables, les volumes persistent.
Comment déploieriez-vous en production ?	Chaque service a déjà son Dockerfile : déploiement conteneurisé (Render, Railway...), bases managées (Supabase, Atlas), secrets dans les variables d'environnement de la plateforme, ALLOW_DEV_LOGIN=false et HTTPS partout.